Learning Impulse Control in Flappy Bird with Proximal Policy Optimization: Mathematical Analysis and Reproducible Experiments

Prashanna Tiwari

24 October 2026

Abstract

This report develops a reproducible Flappy Bird game and a reinforcement learning controller trained with clipped Proximal Policy Optimization (PPO). A single simulator supplies headless training, evaluation, and interactive play. We specify the discrete impulse dynamics, obstacle process, collision geometry, observation map, rewards, and episode boundaries, then connect these choices to actor–critic optimization. Particular attention is given to bootstrapping at external time limits, preserving collision information during pipe transitions, and distinguishing a sampled training policy from greedy deployment. The mathematical development includes a closed-form impulse arc, a proof that the induced observation process is Markov, an exact horizon score ceiling, the likelihood-ratio and baseline identities for the policy gradient, the generalized advantage estimator as a weighted family of n-step returns, and a closed-form characterization of the clipped surrogate and of the approximate-KL stopping statistic. Three independent training seeds each receive 1,003,520 interaction frames. Checkpoints are selected on ten validation courses; evaluation uses thirty separate pipe seeds. The reference greedy policy averages 67.73 pipes and reaches the 6,000-frame limit in 96.7% of test episodes. Across the three trained controllers, the mean test score is 67.91, with sample standard deviation 0.15 between seed means. Because scores are censored by the frame cap, survival fractions are reported with exact Clopper–Pearson intervals and paired controller differences with exact sign and McNemar tests. Episode-level data, model weights, diagnostic curves, software tests, and reproduction commands accompany the report. These results describe this structured-observation simulator; they do not establish performance on screenshots, the original commercial game, or arbitrary obstacle distributions.

1. Problem and implementation scope

Flappy Bird is an impulse-control problem. The player chooses when to replace the bird's vertical velocity with an upward impulse while gravity accelerates it downward. Horizontal motion is represented by pipes translating toward a bird with fixed horizontal position. A successful controller must anticipate the next opening, remain inside it throughout horizontal overlap, and recover before the following obstacle.

The repository initially contained Double DQN code, five-dimensional observations, and separate implementations of training and playable physics. The completed implementation uses an eight-

dimensional observation and a categorical PPO actor with a scalar critic. Original code is retained in `legacy/`; original DQN checkpoints are separate from the versioned PPO checkpoint. This report evaluates only the new shared simulator. Comparing its scores directly with the historical DQN log would confound the learning algorithm with environment changes.

The project has four coupled outputs. `flappyenv.py` defines the environment; `ppo.py` and `train_net.py` implement learning; `main.py` and `renderer.py` provide playable manual and AI modes; `evaluate.py` and `scripts/report.py` produce evaluation records and the paper's figures. Rendering never changes the transition function. Consequently, changing playback speed changes wall-clock presentation, while one action always advances exactly one physics frame.

The scientific question is whether a small policy trained from structured geometric observations can acquire reliable timing on randomized courses. This is an implementation study, not a claim of a new reinforcement learning algorithm.

2. Simulator as a stochastic control process

2.1 State, actions, and obstacle generation

Let the bird's vertical center and velocity at frame t be y_t and v_t. Downward is positive. Its horizontal center is fixed at $x_b = 80$ pixels and its collision half-width is $r = 15$ pixels. The screen measures $W = 800$ by $H = 600$ pixels. A pipe is described by its left edge $x_t^{(i)}$, upper opening boundary $h^{(i)}$, and lower opening boundary $b^{(i)} = h^{(i)} + G$. Its width is $w = 60$ and its opening height is $G = 150$ pixels.

The action space is $\mathcal{A} = \{0, 1\}$. Action zero coasts; action one flaps. Opening heights are integer-valued independent draws,

$$h^{(i)} \sim \text{Uniform}\{80, 81, \dots, 370\}.$$

An episode starts at $y_0 = 300$, $v_0 = 0$, with initial pipe left edges $350, 610, 870$. The simulator maintains at least three pipes whose trailing edges have not yet cleared the bird's left collision boundary. New pipes appear one spacing $D = 260$ beyond the last pipe. Fully offscreen pipes are removed. A local seeded NumPy generator makes each environment's obstacle sequence reproducible and independent of unrelated environment resets.

For mathematical analysis, random draws are treated as independent samples rather than as the finite internal state of a pseudorandom number generator. The complete implementation state also includes the pipe list, pass flags, score, frame count, and generator state. Score is a diagnostic accumulator, not a dynamical input. The 6,000-frame limit is an external evaluation cutoff rather than an intrinsic terminal event in the continuing control task.

2.2 Exact discrete dynamics

With gravitational acceleration $g = 0.5$ pixels per frame squared, upward reset velocity $u = -8$, and downward speed cap $v_{\max} = 10$, the transition is

$$\tilde{v}_t = (1 - a_t)v_t + a_t u, \qquad v_{t+1} = \min(\tilde{v}_t + g, v_{\max}),$$

$$y_{t+1} = y_t + v_{t+1}, \qquad x^{(i)}_{t+1} = x^{(i)}_t - c, \qquad c = 3.$$

This is a semi-implicit Euler update: acceleration changes velocity before the position update. A flap therefore moves the bird by -7.5 pixels on its first frame. Repeated action one resets velocity every frame; it does not accumulate upward impulse. Physics constants are measured in frame units. The nominal display rate of 60 frames per second is a presentation convention.

The vertical subsystem admits an exact closed form, which fixes the timing scale of the control problem.

Proposition 1 (single-impulse arc). Fix a frame t_0 at which the bird flaps, write $y_0 = y_{t_0}$ and $v_0 = v_{t_0}$, and suppose no further flap occurs. If the speed cap is inactive on frames $t_0 + 1, \dots, t_0 + n$, then

$$v_{t_0+k} = u + gk = -8 + 0.5k, \qquad y_{t_0+n} - y_0 = \sum_{k=1}^{n} (u + gk) = 0.25n^2 - 7.75n.$$

Proof. The flap replaces the incoming velocity by u, so the recurrence $v_{k+1} = v_k + g$ with $v'_0 = u$ gives $v_k = u + gk$. The position update is applied after the velocity update, so the displacement of frame k is exactly v_k. Summing, $\sum_{k=1}^{n}(u + gk) = un + gn(n+1)/2$. Substituting $u = -8$ and $g = 1/2$ yields $-8n + 0.25n^2 + 0.25n = 0.25n^2 - 7.75n$. The cap is inactive for $k < 36$ because $u + gk < 10$ there, which covers the whole arc below. □

The first displaced frame moves the bird $u + g = -7.5$ pixels. The quadratic $0.25n^2 - 7.75n$ is minimized at $n^* = 15.5$, so the integer minimizers are $n = 15$ and $n = 16$ with displacement -60; the bird therefore rises at most 60 pixels after a single impulse, and it returns to its starting height at the positive root $n = 31 < 36$, so the speed cap never binds on that arc.

Corollary 1 (impulse-height bound and the shape of the control problem). A single flap lifts the bird by at most 60 pixels, which is exactly half the safe vertical band $G - 2r = 120$, and the maneuver occupies 31 frames, about one third of the inter-pipe transit time $D/c \approx 86.7$ frames. Flapping on m consecutive frames instead displaces $-7.5m$ exactly, since every frame resets the velocity to u before gravity acts. Any admissible trajectory is therefore a superposition of these two regimes, and vertical repositioning is slow relative to obstacle approach. This is why a controller must commit to an impulse well before horizontal overlap begins.

The random baseline is quantitatively predicted by this calculation. Ignoring the cap, if each frame flaps independently with probability p, and K denotes the number of frames elapsed since the most recent flap, then K is geometric with mean $1/p$ and the mean velocity is $u + g/p$. At $p = 1/2$ this is -7 pixels per frame, so the bird should reach the ceiling from $y_0 = 300$ after roughly $285/7 \approx 40.7$ frames; the measured mean episode length is 42.0 frames (Table 1). This approximation is not a model of the learned feedback policy; it explains why equal action probabilities are a poor default flight strategy.

2.3 Collision and scoring geometry

The renderer draws a circular bird, while the simulator deliberately uses its axis-aligned bounding square for collision. This conservative geometry is shared by manual play, training, and evaluation. Boundary contact is fatal when

$$y_{t+1} - r \le 0 \quad \text{or} \quad y_{t+1} + r \ge H.$$

Horizontal overlap with pipe i occurs when

$$x_b + r \ge x_{t+1}^{(i)} \quad \text{and} \quad x_b - r \le x_{t+1}^{(i)} + w.$$

During overlap, safe vertical placement requires the strict inequalities

$$h^{(i)} + r < y_{t+1} < b^{(i)} - r.$$

The safe center interval has width $G - 2r = 120$ pixels. Equality is treated as collision, avoiding disagreement about touching edges. Pipe contact spans approximately $(w + 2r)/c = 30$ frame intervals; with inclusive contact and the initial alignment, the first obstacle overlaps the bird on frames 85 through 115. Clearing the front edge is insufficient: the bird must remain inside the gap until the trailing edge clears its entire collision square.

A pipe is counted once, on a surviving transition satisfying

$$x_{t+1}^{(i)} + w < x_b - r.$$

The `passed` flag prevents duplicate scoring. Death is checked before pass rewards, so a fatal transition never awards a point. This definition measures actual obstacle clearance instead of waiting until a pipe leaves the screen.

2.4 Observation map and informational assumptions

The policy receives the first two pipes whose trailing edges have not cleared the bird's left boundary. Its observation is

$$o_t = \left[\frac{y_t}{H}, \frac{v_t}{10}, \frac{x_t^{(1)} - x_b}{W}, \frac{h^{(1)}}{H}, \frac{b^{(1)}}{H}, \frac{x_t^{(2)} - x_b}{W}, \frac{h^{(2)}}{H}, \frac{b^{(2)}}{H} \right]^\top \in \mathbb{R}^8.$$

The observation is normalized by physical scales, not clipped into $[0, 1]$. Vertical velocity can be negative, horizontal distances can be negative during overlap, and terminal observations can extend beyond the safe vertical interval. `observation_space` therefore permits real-valued components.

Both upper and lower opening boundaries are included for readability even though fixed G makes one redundant. Fixed spacing likewise makes the second horizontal distance predictable from the first. The eight-vector has five independent geometric degrees of freedom under the default configuration: y, v, one horizontal pipe position, and two opening heights. The redundancy avoids requiring the network to reconstruct boundary coordinates from constants.

The second pipe supplies advance information during recovery. A pipe remains the first target until its trailing edge clears the bird's left edge, even if it has already passed the bird's center. Switching earlier would remove an obstacle that can still kill the agent. The third pipe is not observed. Because fresh gap heights are independent and pipes move monotonically, its hidden height has

not been observed previously and does not create a history-dependent prediction under the ideal independent-draw model. Correlated future openings would require revisiting this assumption or adding memory.

2.5 Controlled Markov structure of the observation process

The full simulator state $s_t = (y_t, v_t, \{(x_t^{(i)}, h^{(i)}, b^{(i)}, \text{passed}^{(i)})\}_i, \text{score}, t)$ has a deterministic transition kernel conditioned on fresh obstacle draws, and the natural termination and truncation flags split it into absorbing and cutoff events, so the task is a standard episodic Markov decision process with discount γ. The policy, however, does not observe s_t; it observes $o_t = \phi(s_t) \in \mathbb{R}^8$, the map defined in Section 2.4. A naive reading would therefore classify the problem as a partially observed control problem. The following proposition shows that the observation process is nevertheless Markov under the independent-draw obstacle model.

Proposition 2 (Markov property of the observation process). Under i.i.d. uniform gap heights and deterministic dynamics, for any bounded measurable f,

$$\mathbb{E}[f(o_{t+1}) \mid o_0, a_0, \dots, o_t, a_t] = \mathbb{E}[f(o_{t+1}) \mid o_t, a_t].$$

Proof. Pipes are created at spacing D and all translate at speed c, so the left edges of every pipe still on screen are determined by the first visible left edge, hence by o_t. A pipe is hidden exactly when $x^{(i)} + w < x_b - r$, and such a pipe cannot satisfy the overlap condition $x_b - r \le x^{(i)} + w$; hidden pipes therefore cannot cause a collision, and they cannot influence pipe culling or creation, which depend only on horizontal geometry. The next observation is a deterministic function of the current visible geometry, the action, the bird's velocity, and the gap heights of pipes that become visible on the next transition; those heights are drawn i.i.d. and are independent of $(o_0, a_0, \dots, o_t, a_t)$ given the geometry already in o_t. Score and frame count do not enter ϕ and do not affect the dynamics. Hence the conditional law of o_{t+1} is measurable with respect to (o_t, a_t). □

Two consequences matter for learning. First, a memoryless policy is without loss of Markovity for this obstacle model, so the feed-forward actor of Section 4.1 is not handicapped by its lack of recurrence. Second, the proposition fails as soon as gap heights are correlated in time or depend on history, because then the hidden pipes carry a posterior that depends on the full observation history; that regime would require recurrent or memory-augmented policies and is listed as future work in Section 8.

3. Rewards, return, and the evaluation ceiling

For a living transition passing k_t new pipes and a collision indicator d_t, reward is

$$r_t = \begin{cases} -1, & d_t = 1, \\ 0.1 + 5k_t, & d_t = 0. \end{cases}$$

The training objective uses discount $\gamma = 0.99$:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\tau-1} \gamma^t r_t \right],$$

where τ is natural collision time in the underlying continuing task. Rollouts and practical episodes can stop earlier and bootstrap from a value estimate. The survival term supplies frequent feedback before the agent reaches a pipe. The pipe bonus rewards task progress; the death penalty makes collision locally undesirable.

This reward does not encode exactly the same preference as maximizing undiscounted pipe count. In particular, the survival term is not potential-based shaping with an established policy-invariance guarantee. Different weights or discount factors could favor different behavior. We report both return and pipes rather than treating them as interchangeable. For a capped evaluation of length L, pipe count S, and death flag d, the undiscounted logged return can be independently checked from

$$R = 0.1(L - d) + 5S - d.$$

With fixed spacing and speed, surviving long enough forces pipe encounters, and the scoring rule yields an exact ceiling on any finite horizon.

Proposition 3 (horizon score ceiling). Pipe n has left edge $x^{(n)}(t) = 350 + 260(n-1) - 3t$ while alive, so it clears the bird's collision square on the first surviving transition with $x^{(n)}(t) + w < x_b - r$, that is,

$$t > \frac{345 + 260(n-1)}{3}, \qquad \text{hence} \qquad L_n = \left\lfloor \frac{345 + 260(n-1)}{3} \right\rfloor + 1.$$

The maximum attainable score in a horizon of $L \geq 116$ frames is therefore

$$S_{\max}(L) = \max\{n : L_n \leq L\} = \left\lfloor \frac{3L - 345}{260} \right\rfloor + 1.$$

Proof. Each pipe moves at constant speed $c = 3$, so its position is linear in t; the clearing condition $x^{(n)}(t) + w < 65$ rearranges to $t > (345 + 260(n-1))/3$, whose least integer solution is L_n as stated. Pipes are scored in creation order and at most one score is added per transition, so the score at time L is the largest n with $L_n \leq L$. Solving the inequality for n gives $n \leq 1 + (3L - 345)/260$, and integrality yields the closed form. □

At the standard external limit $L = 6000$, $S_{\max} = 68$; at the extended limit $L = 30000$, $S_{\max} = 345$. A score of 68 is therefore censored by the horizon: it is not evidence that a controller can play indefinitely. For example, a full-length surviving episode has return $0.1(6000) + 5(68) = 940$, and the first possible pass occurs at frame $L_1 = 116$. A longer 30,000-frame evaluation is also recorded to probe behavior beyond the standard cap.

The discount horizon $1/(1-\gamma) = 100$ frames is similar to the roughly $260/3 \approx 86.7$ frames between pipes. By contrast, the advantage trace scale $1/(1 - \gamma\lambda) \approx 16.8$ frames for $\lambda = 0.95$ is shorter. The critic carries information about outcomes beyond the immediate trace; a short advantage trace alone is not the complete planning horizon.

4. Policy, critic, and learning equations

4.1 Parameterization

The actor and critic use separate multilayer perceptrons. Each has input dimension eight, two hidden layers of width 64 with tanh activations, and a linear output. The actor outputs logits $z_\theta(o) \in \mathbb{R}^2$ and defines

$$\pi_\theta(a \mid o) = \frac{\exp z_{\theta,a}(o)}{\sum_{j=0}^{1} \exp z_{\theta,j}(o)}.$$

The critic outputs $V_\phi(o) \in \mathbb{R}$. The actor contains $8 \cdot 64 + 64 + 64 \cdot 64 + 64 + 64 \cdot 2 + 2 = 4866$ parameters; the critic contains 4801, for a combined 9667. No convolutional encoder, recurrence, replay buffer, target network, or demonstration data is used.

Hidden-layer weights receive orthogonal initialization with gain $\sqrt{2}$. Actor and critic output gains are 0.01 and 1.0 respectively. Actor biases start near $[0, -2]$, yielding an approximate initial flap probability $1/(1+e^2) \approx 0.119$. This explicit action prior discourages constant flapping while retaining exploration. It is a design choice rather than a learned fact, and no ablation here establishes its necessity.

4.2 Policy-gradient motivation

Write a trajectory as $\xi = (o_0, a_0, \dots, o_{\tau-1}, a_{\tau-1})$ with probability $p_\theta(\xi) = \prod_t \pi_\theta(a_t \mid o_t)\, \mu(o_{t+1} \mid o_t, a_t)$, where μ is the simulator kernel of Section 2 and does not depend on θ. Differentiating $\log p_\theta(\xi)$ therefore leaves only policy terms, and the likelihood-ratio identity gives

$$\nabla_\theta J(\theta) = \mathbb{E}_\theta \left[\sum_{t=0}^{\tau-1} \nabla_\theta \log \pi_\theta(a_t \mid o_t)\, G_t \right], \qquad G_t = \sum_{k \geq t} \gamma^{k-t} r_k,$$

where J is the discounted objective of Section 3 and G_t is the return from t. The score function has two exact identities that justify the actor–critic form used by the implementation. First, for the softmax policy of Section 4.1, with logits $z_\theta(o)$ and one-hot vector e_a,

$$\nabla_\theta \log \pi_\theta(a \mid o) = \left(\tfrac{\partial z}{\partial \theta}\right)^\top (e_a - \pi_\theta(\cdot \mid o)),$$

so the score is a linear image of the centered one-hot action, its expectation under π_θ vanishes, and its covariance is the Fisher information $\mathrm{diag}(\pi) - \pi\pi^\top$. Second, replacing the return target by any baseline $b(o_t)$ that does not depend on a_t leaves the expectation unchanged, because

$$\mathbb{E}[\nabla_\theta \log \pi_\theta(a_t \mid o_t)\, b(o_t)] = b(o_t) \sum_a \pi_\theta(a \mid o_t) \nabla_\theta \log \pi_\theta(a_t \mid o_t) = b(o_t) \nabla_\theta \sum_a \pi_\theta(a \mid o_t) = 0.$$

Choosing $b = V_\phi$ turns $G_t - V_\phi(o_t)$ into an advantage target and reduces estimator variance. Empirical updates weight $\nabla_\theta \log \pi_\theta(a_t \mid o_t)$ by an estimated advantage. Uniform sampling of rollout transitions is the practical PPO surrogate; we do not claim that this finite-batch update

exactly evaluates the full discounted trajectory gradient, which would require summing over all starting states weighted by their discounted occupancy.

4.3 Advantage computation and episode masks

Generalized advantage estimation aggregates discounted temporal-difference residuals [2]. For the project's batched trajectories, define d_t as natural termination and q_t as external truncation. The implementation uses

$$\delta_t = r_t + \gamma(1 - d_t)V_{\phi_{\text{old}}}(o_{t+1}) - V_{\phi_{\text{old}}}(o_t),$$

$$\widehat{A}_t = \delta_t + \gamma\lambda(1 - \mathbf{1}\{d_t \vee q_t\})\widehat{A}_{t+1}, \qquad \widehat{R}_t = \widehat{A}_t + V_{\phi_{\text{old}}}(o_t).$$

The final trace carry at a rollout boundary is zero. Its residual still contains the next-state value. Death removes bootstrapping; truncation keeps it but stops the trace from crossing a reset [3]. Next-state values are computed from observations saved **before** resetting an environment. For a truncated transition with $r = 1$, $V(o) = 2$, $V(o') = 10$, and $\gamma = 0.9$, the residual is 8; for natural death with those values it is -1. Tests verify both cases and prevent leakage from a subsequent episode.

Proposition 4 (GAE as a discounted family of n-step returns). On a boundary-free segment the recursion is equivalent to

$$\widehat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}, \qquad \text{and} \qquad \widehat{A}_t = (1 - \lambda) \sum_{k \geq 1} \lambda^{k-1} (R_t^{(k)} - V_{\phi_{\text{old}}}(o_t)),$$

where $R_t^{(k)} = \sum_{l=0}^{k-1} \gamma^l r_{t+l} + \gamma^k (1 - d_{t+k-1}) V_{\phi_{\text{old}}}(o_{t+k})$ is the k-step return with the same terminal masking, and both series truncate at the end of the stored segment.

Proof. Unrolling $\widehat{A}_t = \delta_t + \gamma\lambda\widehat{A}_{t+1}$ from a zero carry at the segment end gives the first identity. For the second, note that the k-step return telescopes as $R_t^{(k)} - V(o_t) = \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}$ when the terminal mask suppresses the trailing bootstrap term, and retains a single boundary value term when it does not; in either case, substituting the k-step returns into the right-hand side and exchanging the order of summation, the coefficient of δ_{t+l} is

$$(1 - \lambda) \sum_{k \geq l+1} \lambda^{k-1} \gamma^l = \gamma^l \lambda^l = (\gamma\lambda)^l,$$

using $\sum_{j \geq l} \lambda^j = \lambda^l / (1 - \lambda)$. The masks make every term past a death or a rollout boundary vanish, so both series are finite in implementation. □

This identity makes the bias–variance trade-off explicit: $\lambda = 0$ recovers one-step temporal-difference advantages, which are cheap but inherit critic bias; $\lambda = 1$ recovers undiscounted Monte-Carlo advantages within the segment, which are unbiased given a correct critic at the boundary but have the largest variance. The chosen $\lambda = 0.95$ sits near the high-λ end, consistent with a short effective trace $1/(1 - \gamma\lambda) \approx 16.8$ frames against a discount horizon of 100 frames.

Before optimization, advantages are centered and divided by their population standard deviation plus 10^{-8}. Returns remain on their original reward scale and are treated as fixed targets. Batch normalization of advantages couples finite-batch samples; it is a numerical heuristic rather than a claim of an unchanged finite-sample estimator. Note that centering subtracts the batch mean, a constant with respect to θ, so by the baseline identity of Section 4.2 it does not change the expected policy-gradient direction on the same batch.

4.4 Clipped optimization

PPO compares the current and rollout policies through an action-probability ratio [1]:

$$\rho_t(\theta) = \exp[\log \pi_\theta(a_t \mid o_t) - \log \pi_{\theta_{\text{old}}}(a_t \mid o_t)].$$

The actor loss is

$$\mathcal{L}_\pi = -\frac{1}{B} \sum_t \min \left[\rho_t \overline{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \overline{A}_t \right], \quad \epsilon = 0.2.$$

Here $\overline{A}$ denotes normalized advantages. Positive-advantage samples stop gaining clipped-surrogate benefit above $1+\epsilon$; negative-advantage samples stop gaining benefit below $1-\epsilon$. The next proposition gives a closed form for the per-sample term.

Proposition 5 (closed form of the clipped surrogate). For a single sample with advantage $\overline{A} \neq 0$,

$$\min[\rho\, \overline{A},\ \text{clip}(\rho, 1-\epsilon, 1+\epsilon)\, \overline{A}] = \begin{cases} \overline{A} \cdot \min(\rho, 1+\epsilon), & \overline{A} > 0, \\ \overline{A} \cdot \max(\rho, 1-\epsilon), & \overline{A} < 0. \end{cases}$$

Proof. If $\overline{A} > 0$ the minimum of the two products is $\overline{A}$ times the minimum of ρ and $\text{clip}(\rho)$; since $\text{clip}(\rho) = \min(\max(\rho, 1-\epsilon), 1+\epsilon)$ one checks on the three regions $\rho < 1-\epsilon$, $1-\epsilon \leq \rho \leq 1+\epsilon$, $\rho > 1+\epsilon$ that $\min(\rho, \text{clip}(\rho)) = \min(\rho, 1+\epsilon)$. For $\overline{A} < 0$ the inequality reverses and $\max(\rho, \text{clip}(\rho)) = \max(\rho, 1-\epsilon)$. □

Consequences follow immediately. A sample with $\overline{A} > 0$ receives zero policy gradient once $\rho > 1+\epsilon$, and a sample with $\overline{A} < 0$ receives zero policy gradient once $\rho < 1-\epsilon$; in both cases the objective is locally flat in ρ, so the update cannot further increase the probability of an already-boosted action or further decrease an already-suppressed one. When $|\rho - 1| \leq \epsilon$ the clipped and unclipped surrogates coincide, so the two objectives agree to first order at $\theta = \theta_{\text{old}}$, where $\rho = 1$ and $\nabla_\theta \rho = \nabla_\theta \log \pi_\theta(a_t \mid o_t)$.

The implementation minimizes

$$\mathcal{L} = \mathcal{L}_\pi + 0.5 \left[\frac{1}{2B} \sum_t (V_\phi(o_t) - \widehat{R}_t)^2 \right] - 0.01 \left[\frac{1}{B} \sum_t \mathcal{H}(\pi_\theta(\cdot \mid o_t)) \right].$$

The extra factor $1/2$ inside value loss is explicit: the effective mean squared-error coefficient is 0.25. Gradients are clipped to norm 0.5, and Adam uses numerical epsilon 10^{-5}. Logged diagnostics

include entropy, approximate KL, clipped-ratio fraction, value loss, actor loss, and gradient norm before clipping.

For this implementation, a minibatch stops the remaining update epochs if

$$\widehat{D} = \frac{1}{B}\sum_t [(\rho_t - 1) - \log \rho_t] > 0.03.$$

Lemma 1 (the stopping statistic estimates old-to-new KL). Let $a \sim \pi_{\theta_{\text{old}}}(\cdot \mid o)$ and $\rho = \pi_\theta(a \mid o)/\pi_{\theta_{\text{old}}}(a \mid o)$. Then $\mathbb{E}[(\rho - 1) - \log \rho] = D_{\text{KL}}(\pi_{\theta_{\text{old}}} \| \pi_\theta)$, and the integrand is nonnegative, vanishing only at $\rho = 1$.

Proof. $\mathbb{E}[\rho] = \sum_a \pi_{\theta_{\text{old}}}(a \mid o)\, \pi_\theta(a \mid o)/\pi_{\theta_{\text{old}}}(a \mid o) = 1$, so $\mathbb{E}[\rho - 1] = 0$, while $\mathbb{E}[\log \rho] = \sum_a \pi_{\theta_{\text{old}}} \log(\pi_\theta/\pi_{\theta_{\text{old}}}) = -D_{\text{KL}}(\pi_{\theta_{\text{old}}} \| \pi_\theta)$. Nonnegativity of $x \mapsto (x - 1) - \log x$ follows from its derivative $1 - 1/x$, which vanishes only at $x = 1$ where the function value is zero. □

Under exact old-policy action sampling at fixed observations the statistic is therefore an unbiased KL estimator; the finite minibatch is a diagnostic approximation, and $\widehat{D}$ averages over a mixture of observation states rather than a single o. The threshold is checked before applying that minibatch's gradient. PPO clipping and this stop rule do not impose a hard global trust region or prove monotonic score improvement, and the estimator is one-sided: it detects only the direction from the frozen rollout policy to the updated policy.

4.5 Data flow and computational cost

Each update collects $T = 256$ steps in $N = 16$ independently seeded environments, for $NT = 4096$ transitions. The actor samples during collection. Saved arrays contain current observations, pre-reset next observations, actions, rewards, old log probabilities, old values, and both ending flags. Next-state values are evaluated in a batch after collection, before network weights change.

The learner shuffles the batch and performs up to six passes with minibatch size 256. There are at most $6 \cdot 16 = 96$ optimizer steps per rollout; the KL guard can reduce that count. Rollout data are discarded after the update. Network inference is batched across environments, although environment physics steps run sequentially in Python; no separate worker processes are required.

For $P = 9667$ network parameters, dense-network arithmetic scales approximately as $O(NTP)$ for collection and $O(KNTP)$ for K update epochs. Rollout storage is $O(NTd)$ for observation dimension d, with additional scalar arrays per transition. This makes CPU execution practical for the structured state. These estimates would change substantially for pixel inputs.

5. Experimental design

5.1 Training and selection protocol

Three independent runs use master seeds 1, 2, and 3. Run seed s initializes Python, NumPy, and PyTorch RNGs; environment i initially receives seed $s + i$. Resets then continue each local generator's stream. Initial environment seeds overlap across runs, so these are different master RNG runs with partly overlapping pipe streams, not completely disjoint training course sets. Learned action sampling, parameter initialization, and trajectory lengths also differ.

Each run requests one million frames. The learner completes whole rollout batches, so actual training is

$$\left\lceil \frac{1{,}000{,}000}{4096} \right\rceil 4096 = 245 \cdot 4096 = 1{,}003{,}520$$

frames. Learning rate begins at 3×10^{-4} and is linearly reduced as $\alpha_u = \alpha_0[1 - (u-1)/245]$ for update u. Its final value is positive, $\alpha_0/245$.

Every 25 updates and at the last update, the policy is evaluated greedily on seeds 5000–5009. `best.pt` maximizes the pair (mean validation score, mean validation return) lexicographically. Exact ties retain the earlier checkpoint. `final.pt` stores the final iterate, and `latest.pt` is saved atomically after each update. The bundled `models/ppo.pt` is the seed-1 validation-selected policy, chosen as the reference run before comparing test performance between training seeds.

The validation panel is repeatedly reused and can be overfit through checkpoint selection. It therefore serves model selection, not the final performance claim. Test courses use seeds 10000–10029 and are not used to select checkpoints or revise training hyperparameters. Each episode has an external limit of 6000 frames. The same test seeds are used for every policy, making course comparisons paired.

5.2 Baselines and deployment rules

The coast baseline always chooses zero. The uniform random baseline chooses each action with probability one half using a seeded generator per episode. The height heuristic finds the first uncleared pipe, computes its gap center $m = (h + b)/2$, and flaps when $y > m + 25$ and $v \geq 0$. The threshold keeps its roughly 60-pixel jump arc around the center of a 150-pixel opening. This controller receives the same underlying geometric information but is hand designed and does not learn.

Greedy PPO chooses $\arg\max_a z_{\theta,a}(o)$. Ties favor action zero. Sampled PPO draws from the learned categorical distribution. These are distinct deployed policies; training always samples. We evaluate sampled behavior for the reference checkpoint separately and seed its PyTorch action RNG at 10000. A different evaluation batching scheme could consume the action RNG in a different order and change the sampled trajectories even with identical pipe seeds.

We record pipe count, undiscounted return, episode length, natural termination, and truncation for every seed. Reaching the cap is called a **time-limit survival** rather than an unlimited-play success. Standard deviations across episodes describe variation over held-out courses for a fixed trained policy. Standard deviation across the three run means describes a different, training-seed source of variation.

5.3 Recorded software environment

Experiments were executed on CPU with one PyTorch thread per process under Python 3.12.3, PyTorch 2.13.0, NumPy 2.5.3, Gymnasium 1.4.0, and Pygame 2.5.2. Matplotlib 3.11.2 generated figures, and pytest 9.1.1 ran tests. Although this PyTorch installation includes CUDA support, training used `device=cpu`. Results are deterministic within the recorded CPU execution setup when software and RNG ordering are held fixed; bitwise identity across hardware or library releases is not asserted.

The checked-in direct dependency snapshot documents these versions. Each training run also writes its arguments, environment constants, PPO constants, and library versions into `config.json`. The reference checkpoint carries its architecture settings and simulator configuration, avoiding manual reconstruction when watching AI play.

5.4 Statistical protocol for censored scores

Every evaluation reports the pipe count S, the undiscounted return, the episode length, and the two ending flags for each of thirty paired courses. Two features of this data shape the analysis.

First, scores are **right-censored** by the frame cap: an episode that reaches the cap contributes $S = S_{\max}(L)$ regardless of how far the controller could actually continue, so the observed score distribution is a censored version of the latent score distribution. We therefore report the uncensored fraction, that is, the time-limit survival rate $\hat{p} = \frac{1}{n}\sum_i \mathbf{1}\{\text{truncated}_i\}$, together with the mean and standard deviation of the censored scores. For a survival count k out of $n = 30$, a two-sided 95% Clopper–Pearson interval is

$$\Big[\, B^{-1}_{\alpha/2}(k, n-k+1),\ B^{-1}_{1-\alpha/2}(k+1, n-k) \,\Big], \qquad \alpha = 0.05,$$

with the endpoints replaced by 0 and 1 when $k = 0$ or $k = n$, where B^{-1} is the beta quantile function. Second, because every controller is evaluated on the same thirty course seeds, comparisons are naturally **paired**. For two controllers with scores $S_i^{(A)}$ and $S_i^{(B)}$ we report the mean paired difference and a percentile bootstrap interval obtained by resampling the n paired differences with replacement 10,000 times. When many paired differences are exactly zero, as happens at a score ceiling, we additionally report the two-sided exact sign test on the nonzero differences, whose p-value is $2\sum_{i=0}^{m}\binom{N}{i}2^{-N}$ truncated at one, with N the number of nonzero differences and m the smaller of the two sign counts. For paired survival indicators we report the exact McNemar test on the discordant pairs, which reduces to the same binomial calculation. These procedures are descriptive for a fixed pair of trained controllers on a fixed course panel; they are not confidence statements over future training runs, and the bootstrap interval cannot be read as resolving differences that the ceiling has made invisible.

6. Results and interpretation

Policy	Mean score	SD	Median	Mean frames	Reached time limit
Coast	0.00	0.00	0.0	38.0	0.0%
Uniform random	0.00	0.00	0.0	42.0	0.0%
Height heuristic	64.60	12.95	68.0	5703.9	93.3%
PPO seed 1 (greedy)	67.73	1.46	68.0	5976.2	96.7%
PPO seed 2 (greedy)	68.00	0.00	68.0	6000.0	100.0%
PPO seed 3 (greedy)	68.00	0.00	68.0	6000.0	100.0%
PPO seed 1 (sampled)	28.77	23.92	23.0	2597.8	16.7%

Table 1 reports thirty held-out courses for every row. Sample standard deviations use denominator $n - 1$. These measurements are generated directly from the episode-level JSON files in `results/`.

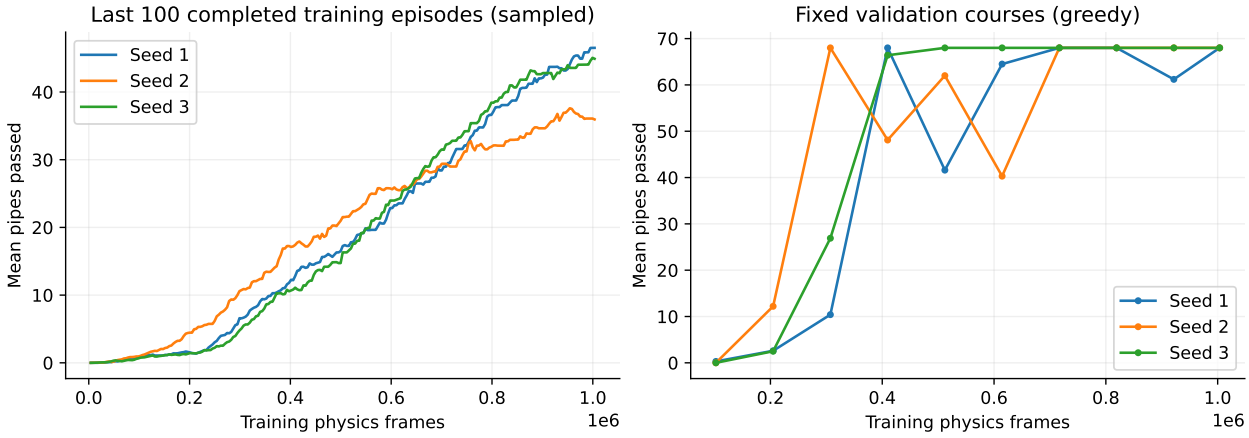


Figure 1: Training and validation curves for three master seeds. Training scores average the last 100 completed sampled episodes; validation uses greedy actions on ten fixed courses. The two panels measure different policies and course sets.

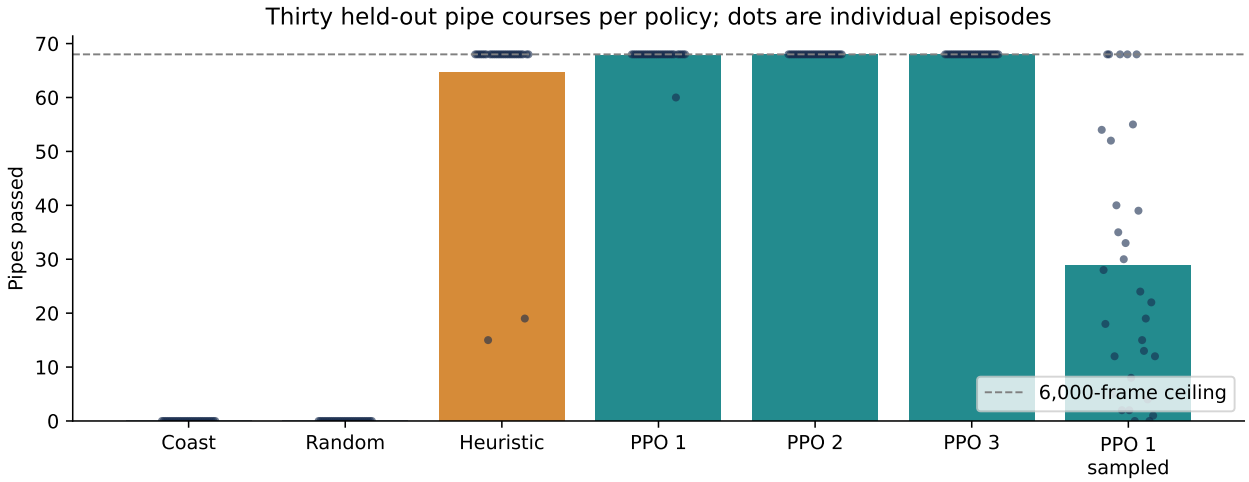


Figure 2: Held-out performance at the 6,000-frame cap. Dots represent individual courses. The dashed line is a simulator-imposed score ceiling, not an estimate of indefinite ability.

The reference greedy PPO policy averages 67.73 pipes and survives to the cap in 96.7% of episodes. Across training-run means, the average is 67.91 with sample standard deviation 0.15. Three runs provide a limited view of training variability; they are insufficient for a broad claim of robustness to initialization.

The uniform random and coast controllers fail before the first pipe. This establishes that surviving requires action timing, but these are weak baselines. The hand-designed controller is a stronger reference because it exploits known geometry and physics. Its competitive performance shows that this task is solvable by a compact feedback rule and that a learned controller should not be presented as necessary for gameplay.

For reference PPO minus the heuristic on paired courses, the mean score difference is 3.13 pipes. A percentile bootstrap resampling the thirty paired score differences 10,000 times gives an approximate 95% interval of [-0.53, 8.43]. This interval describes these courses conditional on two fixed controllers, not uncertainty over training seeds. Its discrete, ceiling-censored data and small panel make it descriptive evidence rather than a definitive significance test. The discreteness is severe: on 27 of the 30 courses both controllers reach the ceiling and the paired difference is exactly zero. Reference PPO scores higher on two courses, by 53 and 49 pipes, and lower on one, by 8 pipes. The two-sided exact sign test on these three nonzero differences returns $p = 1.0$, so this panel provides no distributional evidence that separates the two controllers, and the bootstrap interval above should be read in that light.

Exact 95% Clopper–Pearson intervals for the survival fractions at the standard cap are $[0.83, 1.00]$ for the reference policy (29/30), $[0.88, 1.00]$ for each of seeds 2 and 3 (30/30), and $[0.78, 0.99]$ for the heuristic (28/30). These intervals are wide because $n = 30$ is small; they are consistent with a large difference in latent reliability only in combination with the extended-horizon results below.

The gap between sampled and greedy PPO is consequential. The stochastic policy retains a nonzero probability of poorly timed impulses; repeated opportunities for such actions accumulate over a long episode. Greedy selection can suppress those rare actions and yield much stronger flight. Thus high greedy evaluation scores do not justify claiming that the sampled policy has the same reliability. Both the table and checkpoint documentation explicitly identify the action-selection rule.

Training curves can remain substantially below validation scores for two reasons: training samples actions while validation is greedy, and the last 100 completed episodes can include flights collected over many earlier updates. As flights become longer, that moving window lags the current policy. A train–validation gap here is therefore not by itself evidence of a dataset generalization effect.

Extended evaluations use a 30,000-frame horizon and the same thirty initial course seeds:

Controller	Mean pipes	SD	Mean frames	Reached time limit
Reference PPO (greedy)	330.20	58.70	28716.4	93.3%
Height heuristic	262.37	116.66	22831.6	53.3%

The reference PPO records are in `results/ppo_seed1_long.json`. The heuristic records are in `results/heuristic_long.json`. Changing only the external horizon exposes later obstacles absent from the standard evaluation. At this horizon the maximum score is 345 by Proposition 3. These tests reveal failures hidden by the shorter cap, but surviving episodes remain right-censored. The reference controller's 28 full-length flights out of 30 are evidence of sustained control on these courses, not proof of indefinite survival.

The paired survival table at 30,000 frames is informative in a way the short horizon cannot be: both controllers survive on 14 courses, only the reference policy survives on 14, only the heuristic survives on 2, and neither survives on 0. An exact McNemar test on the 16 discordant pairs, which is a two-sided binomial calculation with $N = 16$ and $m = 2$, gives $p \approx 0.0042$. The corresponding survival intervals are $[0.78, 0.99]$ for the reference policy (28/30) and $[0.34, 0.72]$ for the heuristic (16/30), which no longer overlap at their point estimates. The extended horizon is therefore where the reliability difference between a learned controller and the hand-designed rule becomes measurable; at the standard cap the ceiling hides it.

7. Verification and reproducibility

7.1 Correctness checks

The automated suite checks the Gymnasium environment contract; equality of same-seed trajectories; independence from unrelated resets; the first flap's exact velocity and displacement, including the 60-pixel arc of Proposition 1; boundary death; external truncation; one-time trailing-edge scoring; collisions with a pipe behind the bird's center; observation validity over repeated pipe switches; and the in-gap clearance band of the heuristic controller over the thirty test courses. RGB-array rendering is checked under the dummy SDL video driver.

Learning tests calculate small advantage traces by hand, distinguish death from truncation, check that a reset cannot leak a later episode's reward into a prior trace, verify finite optimization with constant advantages, confirm that a PPO update changes parameters, and round-trip the checkpoint's actor and critic outputs. A short end-to-end CLI run additionally exercises collection, optimization, evaluation, saving, loading, and continued training into a separate run directory.

These tests check implementation invariants. They do not prove policy optimality, correctness for every nondefault configuration, or statistical validity of large-scale reinforcement learning conclusions. Headless display smoke tests exercise window initialization and drawing; interactive keyboard responsiveness still depends on the user's display environment.

7.2 Reproduction commands

From the repository root, create a Python virtual environment and install `requirements-dev.txt`. The generic environment need not inherit system packages. The experiment machine reused its existing CPU-capable PyTorch installation.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-dev.txt
python -m pytest -q
python main.py
python main.py --ai --model models/ppo.pt
```

For each $s \in \{1, 2, 3\}$, train into a new output directory and evaluate the validation-selected checkpoint:

```
python train_net.py --seed 1 --total-timesteps 1000000 --output runs/ppo
python train_net.py --seed 2 --total-timesteps 1000000 --output runs/ppo_seed2
python train_net.py --seed 3 --total-timesteps 1000000 --output runs/ppo_seed3
python evaluate.py --model runs/ppo/best.pt --output results/ppo_seed1.json
```

```
python evaluate.py --model runs/ppo_seed2/best.pt --output results/ppo_seed2.json
python evaluate.py --model runs/ppo_seed3/best.pt --output results/ppo_seed3.json
```

The full `scripts/reproduce.sh` also evaluates baselines and sampled behavior, copies training histories, records extended-horizon results, regenerates figures, and rebuilds this Markdown source. `scripts/build_paper.sh` converts it to LaTeX and PDF using Pandoc and Tectonic. Experiment outputs are included, so reading or rebuilding the paper does not require retraining.

`--resume` restores model and optimizer state into a fresh run with freshly seeded environments and a new learning-rate schedule. It is intentionally a continuation from learned parameters rather than a bit-for-bit mid-trajectory restart: active environment states, rollout buffers, and RNG states are not serialized. Resumption requires matching algorithm and physics settings and a different output directory to protect existing logs.

On the experiment machine, unrelated system-wide ROS pytest plugins were incompatible with the installed pytest version. Tests were run with `PYTEST_DISABLE_PLUGIN_AUTOLOAD=1`; this only disables third-party pytest plugin discovery and does not alter project behavior. A clean virtual environment normally needs no such setting.

8. Limitations and further experiments

The observation exposes exact positions and velocities. Learning from screenshots would require visual preprocessing, temporal information, a different architecture, and a much larger interaction budget. The renderer is not a sensor for this policy. Training and evaluation share one simulator, so no transfer to a separate Flappy Bird implementation is demonstrated.

Gap heights vary, but pipe speed, spacing, gap size, gravity, and flap strength remain fixed. A policy can exploit those constants. Distribution-shift evaluation should vary each quantity while keeping the task physically feasible. Correlated gap sequences would test whether two visible pipes remain sufficient or whether recurrence is useful.

No ablation isolates the second pipe, action prior, reward terms, entropy coefficient, KL threshold, GAE setting, or PPO clipping. Consequently, the report does not attribute the observed performance to any one design choice. A fair PPO–DQN comparison would retrain both algorithms in this same environment with matched observation access, environment-step budgets, checkpoint selection, and multiple master seeds.

The thirty-course panel and three training runs are modest. Ceiling scores obscure performance differences, and reusing one validation panel creates selection bias. Larger test panels, fresh obstacle seeds, uncapped survival analysis, and more training seeds would strengthen the empirical case. Bootstrap intervals conditional on fixed policies must not be interpreted as a confidence interval over all future training runs.

The training objective is a discounted shaped return while evaluation emphasizes undiscounted pipe count and capped survival. Alternative reward weights may improve or harm score without a predictable monotonic relationship. Long-horizon failure probability also compounds even when per-frame error probability is small. If a hypothetical controller had an independent catastrophic-action probability p each frame, then survival for L frames would have probability $(1-p)^L \approx e^{-pL}$, the episode length would be geometric with mean $1/p$ frames, and achieving a survival probability of at least $1-\beta$ over the standard horizon would require

$$p \le 1 - \beta^{1/6000} \approx \frac{-\ln \beta}{6000},$$

which is $p \le 1.7 \times 10^{-6}$ per frame for $\beta = 0.01$ and $p \le 3.4 \times 10^{-7}$ for the extended horizon. Real policy errors are state dependent and correlated, so this is an illustration of accumulated risk, not an estimated model of the measured trajectories. It does explain why a policy with strong short-horizon statistics can still fail occasionally at thirty times the horizon, and why reporting only a mean score conceals the quantity of interest.

A useful next investigation is to compare the final and validation-selected policies under both sampled and greedy action selection, using an independent action RNG per environment. Another is to evaluate safety margins directly: distance to the nearest opening boundary during overlap, number of consecutive flaps, and recovery time after large changes in opening height. Those diagnostics would explain failure mechanisms beyond a single score.

9. Conclusion

The completed system provides a playable game, a custom PPO learner, versioned trained weights, reproducible experiments, and a mathematical specification of their connection. Its strongest engineering property is that training and gameplay use the same transition and collision code. Its strongest empirical evidence is the measured held-out performance of the greedy learned controllers. Its central qualification is that greedy deployment, sampled behavior, and finite-horizon survival answer different questions. Keeping those distinctions explicit makes the implementation and results inspectable and supplies a sound basis for larger experiments.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. *Proximal Policy Optimization Algorithms*. arXiv:1707.06347, 2017. Original paper.

[2] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. ICLR 2016; arXiv:1506.02438, first submitted 2015. Original paper.

[3] Farama Foundation. *Handling Time Limits*. Gymnasium documentation, accessed 24 October 2026. Official documentation.

Appendix A. Default hyperparameters

Setting	Value
Observation dimension / action count	8 / 2
Actor / critic hidden layers	64, 64 each
Activation	tanh
Environments / rollout steps	16 / 256
Frames per update	4096
Requested / actual frames per run	1,000,000 / 1,003,520
Discount γ / trace λ	0.99 / 0.95

Setting	Value
PPO ratio clip ϵ	0.2
Initial learning rate	0.0003
Adam epsilon	0.00001
Maximum update epochs	6
Minibatch size	256
Entropy coefficient	0.01
Value-loss coefficient (with half MSE)	0.5
Gradient norm bound	0.5
Approximate KL stop threshold	0.03
Validation interval	25 updates plus final
Validation / test episode count	10 / 30
Standard episode frame cap	6000
Extended test frame cap	30000

Appendix B. Checkpoint and data schema

A version-1 checkpoint stores `algorithm="PPO"`, `format_version=1`, actor–critic state, Adam state, `ppo_config`, `env_config`, cumulative `global_step`, and master `seed`. Validation-selected checkpoints also include the per-course validation report. Weights-only PyTorch loading is used; unversioned historical DQN weights are incompatible with this architecture.

Every evaluation JSON contains policy name, initial seed, episode count, environment constants, mean score, sample score standard deviation, median and extreme scores, mean length, mean return, cap-survival fraction, checkpoint path, training-step count, and individual episode records. Individual records preserve seed, score, return, length, `terminated`, and `truncated`. Training CSVs preserve one row per update, numerical diagnostics, rolling training statistics, total interaction count, throughput, and periodic validation summaries. `summary.json` and both figures can be regenerated from these records without loading a neural network.

Appendix C. Supplementary derivations

C.1 Clearance band of the height heuristic

The heuristic of Section 5.2 is the strongest non-learned controller in the study, and its design constant can be derived rather than tuned blindly. Two facts about the dynamics are used. First, the flap action sets the velocity to u and gravity then acts, so the velocity after any flap frame is $u + g = -7.5$; since coasting can only increase the velocity, the invariant $v \geq -7.5$ holds on every trajectory. Second, by Proposition 1 the total vertical displacement of a flap maneuver, including its first frame, is at most 60 pixels upward.

Proposition 6 (in-gap clearance band). Fix a gap with center $m = (h + b)/2$ and suppose the target pipe does not change during horizontal overlap. If at the first overlapping frame the bird's center satisfies $y \in (m - 35, m + 35]$, then under the rule "flap if and only if $y > m + 25$ and $v \geq 0$" the center remains in $(m - 35, m + 35]$ for the whole overlap, and therefore maintains a clearance of at least 25 pixels from both fatal boundaries $h + r$ and $b - r$.

Proof. Consider a state with $y \leq m + 25$ and $v \geq 0$. No flap is issued, the frame displaces the bird

by $\min(v+g, v_{\max}) \le 10$, so the next center is at most $m+35$. If that next center exceeds $m+25$, the velocity is still nonnegative, so the following frame flaps and displaces the bird by -7.5, pulling it back below $m+35$; hence $\sup y \le m+35$. Every state with $v<0$ lies on the rising arc of a flap issued at some center $y_{\text{flap}} \in (m+25, m+35]$, so by the arc bound of Proposition 1 it satisfies $y > y_{\text{flap}} - 60 > m-35$ and $y \le y_{\text{flap}} \le m+35$. Substituting $m=(h+b)/2=h+75$, $r=15$, $b=h+150$ gives $m-35=h+40=(h+r)+25$ and $m+35=h+110=(b-r)-25$. □

The threshold 25 is the unique value that equalizes the two clearances just derived: with a trigger threshold τ the band becomes $(m+\tau-60,\ m+\tau+10]$, whose clearances from $h+r=m-60$ and from $b-r=m+60$ are τ and $50-\tau$ respectively, and these coincide only at $\tau=25$. The proposition is deliberately single-gap. When the target pipe changes at a gap transition, m jumps and the bird can be outside the new band with residual velocity; that transition, not the in-gap regulation, is the natural failure mode of the heuristic and is consistent with its nonzero failure count in Table 1.

The hypothesis and the conclusion are both checkable in simulation. Running the heuristic on the thirty test courses produces 59,412 frames of horizontal overlap, of which 35 lie outside the band, and every one of those 35 frames occurs within a few frames of a gap transition while the bird is rising back toward the new center. No episode leaves the band after entering it. The check is automated in `tests/test_heuristic_band.py`, together with a direct verification of the -7.5-per-frame flap displacement and the 60-pixel arc of Proposition 1.

C.2 Arithmetic of the networks and of one training run

Each actor layer has $8{\cdot}64{+}64$, $64{\cdot}64{+}64$, and $64{\cdot}2{+}2$ weights and biases, totalling $576{+}4160{+}130 = 4866$ parameters; the critic's last layer is $64\cdot 1+1$, giving $576+4160+65=4801$. One observation therefore costs $512+4096+128=4736$ multiply–accumulates in the actor and 4672 in the critic, about 9.4×10^3 MACs. A full update evaluates $NT=4096$ observations once for collection, twice more for the bootstrap values, and up to 6 epochs of forward and backward passes (backward at roughly twice the forward cost), for approximately

$$4096 \cdot 9408 \cdot (2+6\cdot 3) \approx 7.7\times10^8 \text{ MACs}$$

per rollout. Over 245 updates this is about 1.9×10^{11} MACs per training run, which is why a single CPU core completes a million frames in the reported wall-clock range and no accelerator is required for structured state of this size.

C.3 Initial action prior

With output weights of gain 0.01 the initial logits are dominated by the bias $[0,-2]$, so the initial flap probability is $\sigma(-2)=1/(1+e^2)\approx0.119$ and the initial policy entropy is

$$\mathcal{H} = -0.119\ln 0.119 - 0.881\ln 0.881 \approx 0.365 \text{ nats},$$

about 53% of the maximum $\ln 2 \approx 0.693$. The prior therefore biases exploration toward coasting, which Proposition 1 and the random-baseline calculation show to be the safer default, while retaining a flap frequency that is high enough to keep both actions in every minibatch.